



Mediacrest Training College

Nairobi, Kenya

Advanced Data Analytics & Machine Learning

8-Week Advanced Training Program

Comprehensive Session Notes

Week 1 · Session 3

Missing Data Mechanisms & Imputation Strategies in MOH Records

Course Trainer: Victor Wandera Lumumba

Role: Statistician, Data Analyst & BI Specialist

Session Date: 13 August 2026

Mode: Hybrid – Online (6–8 PM) / Physical (9–11 AM EAT)

“Classify the mechanism first, then match the treatment — never impute blindly.”

About This Document

These comprehensive notes accompany **Week 1, Session 3** of the **Advanced Data Analytics & Machine Learning** program. While Session 1 and 2 focused on business data quality and basic profiling, this session elevates your skills to *clinical epidemiology and advanced statistical inference*. We transition from handling generic business CSVs to analyzing a real-world Ministry of Health (MOH) facility dataset containing 4,999 patient records.

In clinical and epidemiological research, missing data is not merely an inconvenience; it is a fundamental threat to internal and external validity. Mishandling missing laboratory results or vital signs can lead to biased treatment effect estimates, incorrect policy recommendations, and ultimately, the misallocation of scarce healthcare resources across Kenya’s 47 counties.

Course at a Glance

	8 Weeks (20+ intensive sessions)
Duration	
Core Tools	Python (pandas, NumPy, SciPy, scikit-learn, statsmodels), Power BI
Focus	Deep statistical modelling, supervised ML, missing data theory, causal inference
Capstone	Production-ready ML system with diagnostics, explainability & policy recommendations
Kenyan Context	Healthcare (MOH), agriculture, finance, public policy (CBK, KRA, County Govts)

Where This Session Fits

Week 1 is titled “**Advanced Data Preparation, Data Visualization & Statistical Thinking.**” It is split into three progressive sessions:

- Session 1:** Advanced Data Auditing & Business Data Cleaning.
- Session 2:** Outlier detection, feature encoding, scaling & normalisation.
- Session 3 (this document):** Missing Data Mechanisms (MCAR, MAR, MNAR), Data Leakage Prevention, and Advanced Imputation Strategies (KNN, MICE).

How to Use These Notes

- Follow along in code.** Every listing is runnable. Open the companion Jupyter Notebook (*Week-1-day-3-Data Auditing and Missingness Mechanism.ipynb*) and execute each cell as you read.
- Boxes matter.** **Key Concept** boxes hold the theory; **Warning** boxes hold the mistakes that fail models; **Kenyan Context** boxes connect the material to MOH/CBK/KRA/County work; **Mathematical Foundation** boxes provide the rigorous statistical proofs.

- **Do the exercises.** Each chapter ends with practice tasks. The final empirical comparison is graded.

Contents

About This Document	1
1 Introduction to Clinical Data & The Missing Data Problem	5
1.1 The Reality of MOH Datasets	5
1.2 The Hypertension Prediction Task	5
1.3 Load the Required Libraries	5
2 Data Loading & Initial Exploration	7
2.1 Loading the Dataset	7
2.2 Quantifying Missingness	7
2.3 Visualizing the Missingness Landscape	7
3 Theoretical Foundations of Missingness Mechanisms	9
3.1 Mathematical Formulation	9
4 MCAR: Missing Completely At Random	10
4.1 Definition and Clinical Reality	10
4.2 MOH Example: The Power Outage	10
4.3 Statistical Assessment of MCAR	10
5 MAR: Missing At Random	12
5.1 Definition and Clinical Reality	12
5.2 MOH Example: The Rural Centrifuge	12
5.3 Statistical Assessment of MAR	12
5.3.1 Creating the Contingency Table	12
5.3.2 Calculating Proportions	12
5.3.3 The Chi-Square Test	13
5.3.4 Fisher's Exact Test (Robustness Check)	13
6 MNAR: Missing Not At Random	15
6.1 Definition and Clinical Reality	15
6.2 MOH Example: The Critically Ill Patient	15
6.3 The Statistical Challenge	15
6.4 Handling Suspected MNAR	15
7 The Critical Rule: Preventing Data Leakage	17
7.1 What is Data Leakage?	17
7.2 The Mechanics of Leakage in Imputation	17
7.3 The Correct 7-Step Workflow	17
8 Imputation Strategies: Theory & Implementation	18
8.1 Strategy 1: Listwise Deletion (Complete Case Analysis)	18
8.2 Strategy 2: Median Imputation	18
8.3 Strategy 3: KNN Imputation (K-Nearest Neighbors)	19
8.4 Strategy 4: MICE (Multivariate Imputation by Chained Equations)	19

9 Empirical Comparison on the MOH Dataset	21
9.1 Results Analysis	21
9.2 Why KNN Imputation Won for Prediction	22
9.3 The Listwise Deletion Paradox	22
10 Best Practices & Overall Recommendations	23
10.1 Summary Recommendation Matrix	23
10.2 Prediction vs. Inference	23
11 Glossary of Key Terms	25
12 References & Further Reading	25

1 Introduction to Clinical Data & The Missing Data Problem

1.1 The Reality of MOH Datasets

In an ideal world, every patient visiting a Ministry of Health (MOH) facility would have a complete, error-free electronic health record (EHR). Every vital sign would be measured, every laboratory test performed, and every demographic detail accurately captured. In reality, clinical data is notoriously messy, fragmented, and incomplete.

Rural dispensaries in counties like Turkana or Mandera may lack automated sphygmomanometers, leading to missing blood pressure readings. Centralized laboratories in Nairobi or Kisumu may experience reagent stock-outs, resulting in missing creatinine or hemoglobin panels. Furthermore, critically ill patients may die or default on treatment before follow-up labs can be drawn, creating systematic gaps in the data that are deeply intertwined with the patients' actual health status.

The Policy Stakes

When building predictive models for hypertension risk or resource allocation, missing data is not just a statistical nuisance. If we naively drop patients with missing creatinine levels, and those patients happen to be the sickest individuals from under-resourced rural clinics, our model will systematically **understate** the burden of renal disease in those regions. The resulting policy recommendations will divert funding away from the facilities that need it most.

1.2 The Hypertension Prediction Task

For this session, we utilize a synthetic but highly realistic MOH facility dataset (`htn_dat.csv`) comprising **4,999 patient records**. The primary objective is to build a classification model to predict whether a patient has a Systolic Blood Pressure (SBP) of ≥ 120 mmHg, a standard clinical threshold for elevated blood pressure / hypertension risk.

The dataset contains 16 variables, including:

- **Demographics:** `age`, `married`, `male`, `gender`
- **Vitals:** `SBP`, `DBP`, `BMI`
- **Laboratory:** `hgb_centered` (Hemoglobin), `log_creat_centered` (Log-transformed Creatinine)
- **Clinical History:** `adv_HIV`, `arv_naive`
- **Facility:** `urban_clinic` (0 = Rural, 1 = Urban)
- **Target:** `SBP_ge120` (Binary: 1 if $SBP \geq 120$, 0 otherwise)

1.3 Load the Required Libraries

We begin by importing the standard data science stack, alongside specialized libraries for statistical testing and advanced imputation.

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 from scipy import stats
6
7 # Scikit-Learn imports for Imputation and Modeling
8 from sklearn.experimental import enable_iterative_imputer # Required for MICE
9 from sklearn.impute import SimpleImputer, KNNImputer, IterativeImputer
10 from sklearn.linear_model import LogisticRegression
11 from sklearn.model_selection import train_test_split
12 from sklearn.metrics import accuracy_score, roc_auc_score, classification_report
13 import warnings
14 warnings.filterwarnings('ignore')
15 sns.set_theme(style="whitegrid")
```

Listing 1: Loading the required Python libraries

The IterativeImputer Import

Notice the line `from sklearn.experimental import enable_iterative_imputer`. The MICE algorithm (implemented as `IterativeImputer` in scikit-learn) is still considered experimental in the library's API. You **must** import this enabler before you can use the `IterativeImputer` class, otherwise Python will throw an `ImportError`.

2 Data Loading & Initial Exploration

2.1 Loading the Dataset

We load the `htn_dat.csv` file into a pandas DataFrame and perform an initial inspection.

```
1 print("\n[1] Loading MOH Facility Dataset...")
2 df = pd.read_csv('htn_dat.csv')
3 print(f"Total Patient Records: {len(df)}")
4 df.head()
```

Listing 2: Loading and inspecting the MOH dataset

The dataset contains 4,999 rows and 16 columns. A quick visual inspection of the first few rows reveals immediate red flags: NaN (Not a Number) values are scattered across BMI, `hgb_centered`, `adv_HIV`, and `log_creat_centered`.

2.2 Quantifying Missingness

Before applying any treatment, we must quantify the scale of the problem. We calculate the absolute count of missing values for every column that has at least one missing entry.

```
1 print("\nMissing Values Count by Column:")
2 missing_counts = df.isnull().sum()
3 print(missing_counts[missing_counts > 0])
```

Listing 3: Counting missing values by column

Output:

```
BMI                670
married            164
hgb_centered       1394
adv_HIV            1961
log_creat_centered 1538
dtype: int64
```

The Scale of the Problem

Over 39% of HIV status records (`adv_HIV`) and nearly 31% of Creatinine labs (`log_creat_centered`) are missing. If we were to use **Listwise Deletion** (dropping any row with a missing value), we would lose more than half of our entire dataset. This catastrophic loss of statistical power makes naive deletion entirely unacceptable for this analysis.

2.3 Visualizing the Missingness Landscape

Numbers alone do not reveal the *structure* of missingness. Are the missing values scattered randomly, or do they cluster in specific patients or facilities? We use a binary heatmap to visualize the missingness matrix.

```
1 plt.figure(figsize=(10, 6))
2 sns.heatmap(df.isnull(), cbar=False, cmap="viridis", yticklabels=False)
3 plt.title("Missing Data Heatmap (MOH Patient Records)", fontsize=12)
4 plt.xlabel("Clinical Variables")
```



```
5 plt.tight_layout()
6 plt.show()
```

Listing 4: Generating a missing data heatmap

Interpreting the Heatmap:

- **Vertical Bands:** Columns like `adv_HIV` and `log_creat_centered` appear as dense yellow/-green bands, indicating high overall missingness.
- **Horizontal Clustering:** If we see dark horizontal lines crossing multiple columns, it indicates that *specific patients* are missing multiple labs simultaneously. This is a strong indicator of MNAR (e.g., the sickest patients defaulted before any labs were drawn).
- **Random Scatter:** A diffuse, random scatter of missing pixels in the BMI column suggests the missingness might be unrelated to patient characteristics (MCAR).

3 Theoretical Foundations of Missingness Mechanisms

To treat missing data correctly, we must first diagnose *why* it is missing. In 1976, statistician Donald Rubin formalized the missing data framework, categorizing missingness into three distinct mechanisms. Understanding these mechanisms is the bedrock of epidemiological data analysis.

3.1 Mathematical Formulation

Let X be the complete data matrix, which we partition into observed data X_{obs} and missing data X_{mis} . Let R be the missingness indicator matrix, where $R_{ij} = 1$ if the value is observed, and $R_{ij} = 0$ if it is missing.

The probability of missingness is governed by the conditional distribution:

$$P(R \mid X_{obs}, X_{mis}, \theta) \quad (1)$$

where θ represents the parameters of the missingness mechanism.

Rubin's Missing Data Mechanisms

1. **MCAR (Missing Completely At Random):**

$$P(R \mid X_{obs}, X_{mis}, \theta) = P(R \mid \theta)$$

The probability of missingness depends **neither** on the observed data **nor** on the unobserved (missing) data.

2. **MAR (Missing At Random):**

$$P(R \mid X_{obs}, X_{mis}, \theta) = P(R \mid X_{obs}, \theta)$$

The probability of missingness depends **only** on the observed data, and is conditionally independent of the missing data itself.

3. **MNAR (Missing Not At Random):**

$$P(R \mid X_{obs}, X_{mis}, \theta) \text{ depends explicitly on } X_{mis}.$$

The probability of missingness depends on the **unobserved (missing) values themselves**.

Let us explore each mechanism in the context of our MOH dataset.

4 MCAR: Missing Completely At Random

4.1 Definition and Clinical Reality

A variable is MCAR if the probability of it being missing is entirely independent of any patient characteristics, whether measured or unmeasured. The missingness is essentially a random coin flip.

The MCAR Assumption

MCAR is the strongest and most restrictive assumption. If data is truly MCAR, the subset of patients with complete data is a **perfectly representative random sample** of the entire population. Therefore, any statistical analysis performed on the complete cases will be unbiased.

4.2 MOH Example: The Power Outage

Consider the BMI variable in our dataset. **Scenario:** A random, localized power outage at a rural clinic causes the digital weighing scale to fail for a single afternoon. Every patient seen during that window has a missing BMI. Because the power outage was a random environmental event, the patients missing a BMI measurement are no different (in terms of age, disease severity, or gender) from the patients who were weighed. The missingness is independent of the patient's actual weight (X_{mis}) and their other characteristics (X_{obs}).

4.3 Statistical Assessment of MCAR

While we can never definitively *prove* MCAR, we can gather evidence *consistent* with it. The standard approach is to compare the distribution of **other observed variables** between the group with missing data and the group with complete data. If there are no systematic differences, MCAR is plausible.

For BMI, we will compare the **age** of patients with missing BMI versus those with observed BMI using an independent samples T-test.

```
1 bmi_missing = df[df['BMI'].isnull()]['age'].dropna()
2 bmi_observed = df[df['BMI'].notna()]['age'].dropna()
3
4 t_stat, p_value = stats.ttest_ind(bmi_missing, bmi_observed)
5
6 print(f"Result: T-test p-value for Age difference = {p_value:.4f}")
7 if p_value > 0.05:
8     print("Conclusion: Fail to reject null hypothesis. BMI missingness may be
9     MCAR.")
10 else:
11     print("Conclusion: Significant difference. BMI missingness is likely NOT
12     MCAR.")
```

Listing 5: T-test for MCAR assessment on BMI

Output:

Result: T-test p-value for Age difference = 0.3517

Conclusion: Fail to reject null hypothesis. BMI missingness may be MCAR.

The Independent Samples T-Test

The null hypothesis H_0 is that the mean age of the missing group (μ_{mis}) equals the mean age of the observed group (μ_{obs}).

$$t = \frac{\bar{X}_{mis} - \bar{X}_{obs}}{\sqrt{\frac{s_{mis}^2}{n_{mis}} + \frac{s_{obs}^2}{n_{obs}}}} \quad (2)$$

A p-value of $0.3517 > 0.05$ means we lack sufficient evidence to reject H_0 . The age distributions are statistically indistinguishable, supporting the MCAR assumption for BMI.

The Danger of Assuming MCAR

True MCAR is exceptionally rare in clinical data. A non-significant p-value does **not** prove MCAR; it merely indicates that the specific variable you tested (Age) does not explain the missingness. The missingness could still be driven by an unmeasured variable (e.g., patients with mobility issues couldn't reach the scale). Always treat MCAR as a working assumption, not an absolute truth.

5 MAR: Missing At Random

5.1 Definition and Clinical Reality

A variable is MAR if the probability of it being missing depends on **observed data**, but **not** on the unobserved missing value itself.

The MAR Assumption

The name "Missing At Random" is historically misleading. MAR missingness is **not random** in the colloquial sense; it is highly systematic. However, the system driving the missingness is entirely captured by variables we **already have in our dataset**. Because we can observe the drivers of missingness, we can statistically adjust for them.

5.2 MOH Example: The Rural Centrifuge

Consider the `hgb_centered` (Hemoglobin) variable. **Scenario:** Hemoglobin testing requires a centrifuge to separate red blood cells from plasma. Many rural dispensaries in Kenya lack functional centrifuges or reliable electricity to run them. Consequently, hemoglobin results are missing much more frequently in rural clinics (`urban.clinic = 0`) than in urban hospitals (`urban.clinic = 1`).

Crucially, *within* the rural clinics, the decision to draw blood is not dependent on the patient's actual hemoglobin level (which is unobserved); it is purely a logistical constraint of the facility. Because we have the `urban.clinic` variable in our dataset, we can model and adjust for this missingness.

5.3 Statistical Assessment of MAR

To test for MAR, we evaluate whether the missingness indicator is associated with an observed categorical variable. We will use a **Chi-Square Test of Independence** and **Fisher's Exact Test** to compare the proportion of missing hemoglobin across rural and urban clinics.

5.3.1 Creating the Contingency Table

```
1 df['hgb_missing_flag'] = df['hgb_centered'].isnull().astype(int)
2
3 # Create a contingency table
4 table = pd.crosstab(df['urban.clinic'], df['hgb_missing_flag'])
5 print(table)
```

Listing 6: Contingency table for Hemoglobin missingness by Clinic Type

Output:

hgb_missing_flag	0	1
urban.clinic		
0	1842	795
1	1763	599

5.3.2 Calculating Proportions

```
1 proportions = df.groupby('urban_clinic')['hgb_missing_flag'].mean()
2 print(proportions)
```

Listing 7: Proportions of missingness by group

Output: Rural clinics (0) have a 30.15% missing rate, while Urban clinics (1) have a 25.36% missing rate.

5.3.3 The Chi-Square Test

```
1 from scipy.stats import chi2_contingency
2 chi2, p_value, dof, expected = chi2_contingency(table)
3 print("Chi-square statistic:", chi2)
4 print("p-value:", p_value)
```

Listing 8: Chi-Square Test of Independence

Output:

Chi-square statistic: 13.96717242781634

p-value: 0.0001860305748480715

The Chi-Square Statistic

The Chi-Square test evaluates whether two categorical variables are independent.

$$\chi^2 = \sum \frac{(O_i - E_i)^2}{E_i} \quad (3)$$

where O_i is the observed frequency and E_i is the expected frequency under the null hypothesis of independence. A p-value < 0.001 provides overwhelming evidence to reject the null hypothesis. Clinic type and hemoglobin missingness are **highly associated**.

5.3.4 Fisher's Exact Test (Robustness Check)

When sample sizes in specific cells are small, or to provide an exact probability rather than an asymptotic approximation, we use Fisher's Exact Test.

```
1 from scipy.stats import fisher_exact
2 odds_ratio, p_value = fisher_exact(table)
3 print("Odds ratio:", odds_ratio)
4 print("p-value:", p_value)
```

Listing 9: Fisher's Exact Test

Output: Odds Ratio = 0.787, p-value < 0.001.

Interpretation: The odds of hemoglobin being missing in an urban clinic are 0.787 times the odds in a rural clinic. Because missingness is strongly predicted by an **observed** variable (`urban_clinic`), the MAR assumption is strongly supported.

Reporting MAR in Policy Documents

When writing technical reports for the MOH or County Health Management Teams (CHMTs), use precise statistical language:

"A Chi-square test of independence showed a statistically significant association between clinic

type and hemoglobin missingness, $\chi^2(1) = 13.97, p < .001$. The proportion of missing hemoglobin results was 30.15% in rural clinics compared with 25.36% in urban clinics. This indicates that hemoglobin missingness varies significantly by clinic type, providing evidence consistent with a MAR mechanism, since missingness is associated with an observed variable."

6 MNAR: Missing Not At Random

6.1 Definition and Clinical Reality

A variable is MNAR if the probability of it being missing depends on the **unobserved value itself**. This is the most dangerous and intractable missingness mechanism.

The MNAR Trap

Under MNAR, the fact that a data point is missing **is** the data. The missingness mechanism cannot be fully explained by the variables in your dataset because the driving factor is the missing value itself, which you do not possess.

6.2 MOH Example: The Critically Ill Patient

Consider `log_creat_centered` (Creatinine, a marker of kidney function). **Scenario:** Creatinine tests require blood draws and transport to a centralized regional laboratory. Patients with severely failing kidneys (who would have *extremely high* creatinine levels) are often too critically ill, hemodynamically unstable, or bedbound to be transported from a rural dispensary to the central lab. Alternatively, the most critically ill patients may pass away before the lab results can be processed.

Therefore, the highest, most dangerous creatinine values are exactly the ones that are missing. If you impute these missing values using the median of the healthier, stable patients who *were* able to travel to the lab, you will systematically **underestimate** the prevalence of severe renal failure in the population.

6.3 The Statistical Challenge

The Untestable Nature of MNAR

You **cannot** prove MNAR using standard statistical tests on the observed data. By definition, the data required to prove the mechanism is missing. A non-significant p-value in a t-test or chi-square test does **not** mean the data is MNAR; it just means you haven't found the observed variable that explains the missingness. MNAR is established through **clinical domain expertise**, logical deduction, and sensitivity analysis.

6.4 Handling Suspected MNAR

When you suspect a variable is MNAR (like our creatinine or `adv_HIV` variables, where critically ill patients might die before HIV testing), standard imputation methods (Mean, Median, KNN, MICE) will produce biased estimates.

Best Practices for MNAR:

1. **Sensitivity Analysis:** Run your model under different assumptions (e.g., assume all missing creatinine values are in the 95th percentile of severity) and observe how the model's conclusions change.
2. **Missingness Indicators:** Create a binary flag (`is_missing_creatinine`) and include it as a feature in your predictive model. The model can learn that the *act* of missing the lab

is a strong predictor of mortality or severe disease.

3. **Expert Elicitation:** Consult clinicians to define plausible bounds for the missing values.
4. **Transparency:** Explicitly flag the variable as suspected MNAR in all policy reports.

7 The Critical Rule: Preventing Data Leakage

Before we apply any imputation strategy, we must address the most common and fatal mistake in machine learning workflows: **Data Leakage**.

7.1 What is Data Leakage?

Data leakage occurs when information from the **test set** (data the model should never see during training) inadvertently influences the training process. This results in overly optimistic performance metrics during development, followed by catastrophic failure when the model is deployed in the real world.

The Golden Rule of Imputation

ALWAYS split your dataset into training and testing sets **BEFORE** performing any imputation, scaling, or feature selection.

7.2 The Mechanics of Leakage in Imputation

Suppose you decide to use **Mean Imputation** for the **age** variable.

- **The Wrong Way (Leakage):** You calculate the mean age of the *entire* 4,999 patient dataset (e.g., Mean = 52 years). You fill all missing ages with 52. *Then* you split the data into Train and Test. The test set's age distribution has now contaminated the training set's imputation value.
- **The Right Way (No Leakage):** You split the data first. You calculate the mean age *only* using the Training set (e.g., Mean = 50 years). You use 50 to fill missing ages in the Training set, and you *also* use 50 to fill missing ages in the Test set. The Test set's true mean (which might be 55) was kept completely hidden from the training process.

7.3 The Correct 7-Step Workflow

1. **Load** the raw data.
2. **Separate** predictors (X) and outcome (y).
3. **Split** the data into Train and Test sets (stratified).
4. **Fit** the imputer *only* on the Training data (X_{train}).
5. **Transform** the Training data using the fitted imputer.
6. **Transform** the Test data using the *same* fitted imputer.
7. **Train** the model on the imputed Training data and **Evaluate** on the imputed Test data.

Use Scikit-Learn Pipelines

To enforce this workflow and make it mathematically impossible to leak data during cross-validation, wrap your imputers and models in a `sklearn.pipeline.Pipeline`. The pipeline automatically ensures that `fit()` is only called on the training folds.

8 Imputation Strategies: Theory & Implementation

We will now define our target and features, execute the train-test split, and implement four distinct imputation strategies.

```
1 target = 'SBP_ge120'
2 features = ['age', 'DBP', 'BMI', 'hgb_centered', 'log_creat_centered']
3 X = df[features]
4 y = df[target]
5
6 # Drop rows where the target itself is missing (if any)
7 mask = y.notna()
8 X = X[mask]
9 y = y[mask]
10
11 # TRAIN/TEST SPLIT (70/30, Stratified to maintain event rates)
12 X_train, X_test, y_train, y_test = train_test_split(
13     X, y, test_size=0.3, random_state=42, stratify=y
14 )
```

Listing 10: Defining features and executing the stratified split

8.1 Strategy 1: Listwise Deletion (Complete Case Analysis)

Listwise deletion removes any row that has a missing value in *any* of the analysis variables.

```
1 X_train_del = X_train.dropna()
2 y_train_del = y_train[X_train_del.index]
3 X_test_del = X_test.dropna()
4 y_test_del = y_test[X_test_del.index]
```

Listing 11: Listwise Deletion

Pros: Simple, unbiased if data is strictly MCAR.

Cons: Massive loss of statistical power. In our dataset, this reduces the training set from 3,499 to just 2,030 records (a 42% loss). If data is MAR or MNAR, deletion introduces severe selection bias.

8.2 Strategy 2: Median Imputation

Replaces missing values with the median of the observed values in the training set.

```
1 imp_median = SimpleImputer(strategy='median')
2 X_train_med = pd.DataFrame(imp_median.fit_transform(X_train), columns=X.columns)
3 X_test_med = pd.DataFrame(imp_median.transform(X_test), columns=X.columns)
```

Listing 12: Median Imputation

Pros: Retains full sample size; robust to outliers compared to mean imputation.

Cons: Artificially reduces the variance of the variable. Destroys correlations between variables (e.g., the natural correlation between Age and Creatinine is broken because all missing Creatinines are assigned the exact same static value).

8.3 Strategy 3: KNN Imputation (K-Nearest Neighbors)

KNN imputation identifies the k most "similar" patients in the dataset (based on their non-missing features) and uses their values to estimate the missing data.

```
1 imp_knn = KNNImputer(n_neighbors=5)
2 X_train_knn = pd.DataFrame(imp_knn.fit_transform(X_train), columns=X.columns)
3 X_test_knn = pd.DataFrame(imp_knn.transform(X_test), columns=X.columns)
```

Listing 13: KNN Imputation

KNN Distance Metrics

KNN relies on distance calculations to find neighbors. For continuous clinical variables, the Euclidean distance is standard:

$$d(p, q) = \sqrt{\sum_{i=1}^n (p_i - q_i)^2} \quad (4)$$

Because variables like Age (range 18-90) and Creatinine (range 0.5-10.0) have vastly different scales, KNN requires **feature scaling** (Standardization or Min-Max) prior to imputation, otherwise Age will dominate the distance calculation. Scikit-learn's `KNNImputer` handles some internal scaling, but explicit pipeline scaling is best practice.

Pros: Preserves multivariate relationships and local data structures. Highly effective for prediction tasks.

Cons: Computationally expensive ($O(N^2)$ complexity). Impractical for massive national datasets with millions of rows without specialized indexing (e.g., KD-Trees).

8.4 Strategy 4: MICE (Multivariate Imputation by Chained Equations)

MICE (also known as Fully Conditional Specification) treats each variable with missing data as a target in a regression model, using all other variables as predictors. It cycles through the variables iteratively until convergence.

```
1 imp_mice = IterativeImputer(max_iter=10, random_state=42)
2 X_train_mice = pd.DataFrame(imp_mice.fit_transform(X_train), columns=X.columns)
3 X_test_mice = pd.DataFrame(imp_mice.transform(X_test), columns=X.columns)
```

Listing 14: MICE Imputation

The Gibbs Sampler in MICE

MICE uses a Markov Chain Monte Carlo (MCMC) technique called Gibbs sampling. For a dataset with variables X_1, X_2, X_3 , it draws imputations from the conditional posterior distributions:

$$X_1^{(t)} \sim P(X_1 | X_2^{(t-1)}, X_3^{(t-1)}, \theta_1) \quad (5)$$

$$X_2^{(t)} \sim P(X_2 | X_1^{(t)}, X_3^{(t-1)}, \theta_2) \quad (6)$$

$$X_3^{(t)} \sim P(X_3 | X_1^{(t)}, X_2^{(t)}, \theta_3) \quad (7)$$

This iterative process preserves the complex covariance structure of the clinical data.

Pros: The gold standard for statistical inference and epidemiological research. Accounts for uncertainty by generating multiple imputed datasets (in full Bayesian frameworks). Preserves correlations.

Cons: Computationally intensive. Requires careful tuning of the regression models used for each variable.

9 Empirical Comparison on the MOH Dataset

To determine which strategy yields the best predictive model for hypertension risk, we will train a **Logistic Regression** classifier on each of the four imputed datasets and evaluate them using Accuracy and the Area Under the Receiver Operating Characteristic Curve (AUC-ROC).

```

1 datasets = {
2     "1. Listwise Deletion": (X_train_del, y_train_del, X_test_del, y_test_del),
3     "2. Median Imputation": (X_train_med, y_train, X_test_med, y_test),
4     "3. KNN Imputation": (X_train_knn, y_train, X_test_knn, y_test),
5     "4. MICE Imputation": (X_train_mice, y_train, X_test_mice, y_test)
6 }
7
8 results = []
9 for name, (x_tr, y_tr, x_te, y_te) in datasets.items():
10     if len(x_tr) == 0 or len(x_te) == 0:
11         continue
12
13     # Train Logistic Regression
14     lr = LogisticRegression(max_iter=1000, random_state=42, penalty=None)
15     lr.fit(x_tr, y_tr)
16
17     # Predict and Evaluate
18     preds = lr.predict(x_te)
19     proba = lr.predict_proba(x_te)[: , 1]
20     acc = accuracy_score(y_te, preds)
21     auc = roc_auc_score(y_te, proba)
22
23     results.append({
24         'Strategy': name, 'Train_N': len(x_tr), 'Test_N': len(x_te),
25         'Accuracy': acc, 'AUC': auc, 'Age_Coef': lr.coef_[0][0]
26     })
27
28 res_df = pd.DataFrame(results)
29 print(res_df.to_string(index=False))

```

Listing 15: Training and evaluating the models

9.1 Results Analysis

Table 1: Model Performance Comparison Across Imputation Strategies

Strategy	Train N	Test N	Accuracy	AUC-ROC	Age Coef.
1. Listwise Deletion	2,030	858	0.9510	0.7516	0.0610
2. Median Imputation	3,499	1,500	0.9487	0.7664	0.0582
3. KNN Imputation	3,499	1,500	0.9493	0.7718	0.0587
4. MICE Imputation	3,499	1,500	0.9487	0.7686	0.0591

9.2 Why KNN Imputation Won for Prediction

1. **Highest AUC (0.7718):** AUC measures the model's ability to rank a randomly chosen positive instance ($\text{SBP} \geq 120$) higher than a negative instance. KNN achieved the best discrimination.
2. **High Accuracy (94.93%):** Only marginally lower than Listwise Deletion, but achieved using 72% more training data.
3. **Preservation of Local Structure:** Patients with similar Age, DBP, and BMI naturally have similar Hemoglobin and Creatinine levels. KNN exploits these clinical neighborhoods perfectly.

9.3 The Listwise Deletion Paradox

Notice that **Listwise Deletion** achieved the **highest Accuracy (0.9510)** but the **lowest AUC (0.7516)**. Why? By deleting 42% of the dataset, we disproportionately removed the sickest, most complex patients (those with missing labs). The remaining 2,030 patients represent a "clean", healthy, easy-to-classify subset. The model achieves high accuracy simply because the test set it evaluates on is also stripped of complex cases. However, its *discriminatory power* (AUC) collapses because it has lost the nuanced variance required to separate borderline cases.

Never Chase Accuracy Alone

In clinical datasets with class imbalance or informative missingness, Accuracy is a deceptive metric. Always prioritize AUC-ROC, F1-Score, or Precision-Recall curves. A model that looks "perfect" after listwise deletion is almost always suffering from severe selection bias.

10 Best Practices & Overall Recommendations

There is no universally "best" imputation method. The correct choice depends entirely on your **analytical objective** (Prediction vs. Inference) and the **missingness mechanism**.

10.1 Summary Recommendation Matrix

Table 2: Imputation Strategy Selection Guide

Method	Main Advantage	Main Limitation	Best Use Case
Listwise Deletion	Simple; unbiased if strictly MCAR.	Massive data loss; severe selection bias if MAR/MNAR.	MCAR + Massive sample size.
Mean / Median	Fast; easy to implement; retains sample size.	Destroys variance and covariance; biases standard errors.	Quick EDA; simple baseline models.
KNN	Preserves local multivariate structures; excellent for ML.	Computationally expensive ($O(N^2)$); requires scaling.	Predictive modeling on moderate datasets.
MICE	Preserves covariance; accounts for uncertainty; gold standard for inference.	Complex to tune; computationally heavy.	Epidemiological research; MAR clinical data; statistical inference.

10.2 Prediction vs. Inference

- **For Machine Learning Prediction (e.g., deploying a risk-score API):** KNN or advanced tree-based imputation (like XGBoost’s native missing handling) are often superior because they optimize for discriminatory power (AUC) and preserve complex, non-linear interactions between vitals.
- **For Statistical Inference (e.g., estimating the Odds Ratio of Age on Hypertension for an MOH policy paper):** MICE is mandatory. MICE allows you to generate $m = 5$ distinct imputed datasets, run your regression on each, and pool the coefficients using **Rubin’s Rules**. This correctly inflates the standard errors to account for the uncertainty introduced by the missing data, preventing false-positive (Type I) errors.

Final Policy Note

When presenting findings to the Ministry of Health or County Governments, transparency is paramount. If you utilized KNN imputation for a predictive dashboard, state it clearly. If you suspect MNAR in rural creatinine labs, include a **Sensitivity Analysis** appendix showing how the estimated disease burden shifts if the missing values are assumed to be 20% worse

than the observed median. *Audit before you model, and document every assumption.*

11 Glossary of Key Terms

MCAR	Missing Completely At Random. Missingness is independent of both observed and unobserved data.
MAR	Missing At Random. Missingness depends on observed data, but not the missing value itself.
MNAR	Missing Not At Random. Missingness depends on the unobserved missing value itself.
Listwise Deletion	Complete Case Analysis. Dropping any row that contains a missing value.
Data Leakage	The improper transfer of information from the test set to the training set, leading to overoptimistic evaluation.
KNN Imputer	An algorithm that estimates missing values based on the feature similarity of neighboring data points.
MICE	Multivariate Imputation by Chained Equations. An iterative approach that models each variable conditionally on all others.
AUC-ROC	Area Under the Receiver Operating Characteristic Curve. A metric evaluating a model's ability to distinguish between classes across all thresholds.
Stratified Split	A train-test split that preserves the original proportion of the target classes in both subsets.
Rubin's Rules	The mathematical framework for pooling parameter estimates and standard errors across multiple imputed datasets.

12 References & Further Reading

- Rubin, D. B. (1976). *Inference and Missing Data*. Biometrika, 63(3), 581-592.
- van Buuren, S. (2018). *Flexible Imputation of Missing Data* (2nd ed.). CRC Press.
- Little, R. J. A., & Rubin, D. B. (2002). *Statistical Analysis with Missing Data* (2nd ed.). Wiley.
- Scikit-Learn Documentation: `IterativeImputer` and `KNNImputer`.
- Mediacrest Training College — *Advanced Data Analytics & Machine Learning* curriculum (Weeks 1–8).

Advanced Data Analytics & Machine Learning · Week 1, Session 3

Trainer: Victor Wandera Lumumba · 13 August 2026
